WNDRVR
SUPPORT NETWORK

VXWORKS 7 SDK APPLICATION DEVELOPER GUIDE, SR0650

Copyright Notice

Copyright © 2022 Wind River Systems, Inc.

All rights reserved. No part of this publication may be reproduced or transmitted in any form or by any means without the prior written permission of Wind River Systems, Inc.

Wind River, Simics, Tornado, and VxWorks are registered trademarks of Wind River Systems, Inc. Helix, Pulsar, Rocket, Titanium Cloud, Titanium Control, Titanium Core, Titanium Edge, Titanium Edge SX, Titanium Server, and the Wind River logo are trademarks of Wind River Systems, Inc. Any third-party trademarks referenced are the property of their respective owners. For further information regarding Wind River trademarks, please see:

www.windriver.com/company/terms/trademark.html

This product may include software licensed to Wind River by third parties. Relevant notices (if any) are provided for your product on the Wind River download and installation portals:

https://delivers.windriver.com/

https://windshare.usa.windriver.com/

Wind River may refer to third-party documentation by listing publications or providing links to third-party websites for informational purposes. Wind River accepts no responsibility for the information provided in such third-party documentation.

Corporate Headquarters

Wind River
500 Wind River Way
Alameda, CA 94501-1153
U.S.A.
Toll free (U.S.A.): +1-800-545-WIND
Telephone: +1-510-748-4100
Facsimile: +1-510-749-2010

For additional contact information, see the Wind River website:

www.windriver.com

For information on how to contact Customer Support, see:

www.windriver.com/support

VxWorks 7 SDK Application Developer Guide, SR0650

10 July 2020

1. ABOUT THE APPLICATION DEVELOPER

The role of the *Application Developer* is write and test software solutions running in a VxWorks Real-time Operating System (RTOS) environment supplied by a software development kit (SDK). VxWorks supports several types of applications, including DKMs, RTPs, and shared libraries.

SDK (Software Development Kit)

A complete system that is ready for application development. It includes the tools necessary to compile, debug, and test software solutions.

RTP (Real-Time Process)

An RTP is an executable application that runs in user-space as a process. An RTP runs in its own environment and cannot directly interact with other resources not mapped on to it. It is a VxWorks application project.

DKM (Downloadable Kernel Module)

A DKM is an application that runs in kernel-mode as tasks, with full access to the system hardware. It is a VxWorks application project.

Shared library

A shared library is a package of software components that reside in user-space and which can be accessed by multiple applications.

WNDRVR

2. APPLICATION DEVELOPER PREREQUISITES

Prior to using a VxWorks software development kit (SDK), an application developer must obtain information about how to set up the environment. This will either be provided by your SDK provider or be driven by the requirements from your own application environment.

SDK Characteristic	Choices	Notes
Host type	Windows 10 Ubuntu 18.04 LTS Ubuntu 20.04 LTS	If another host type is required, contact your Wind River representative.
Target architecture	ARM PowerPC RISC-V X86	Each SDK is created based on a specific architecture.
Run-time environment	Hardware target QEMU emulator	Setting up and accessing the hardware is beyond the scope of this documentation. Check with your SDK provider to see if QEMU is available.
Debugging	wrdbg Simics	The Wind River Debugger (wrdbg) is the default. Other debugging methods are beyond the scope of this documentation. For information about purchasing Simics, contact your Wind River representative.
Third party add-on features	CMake Python Boost Rust	VxWorks supports a number of optional add-ons that can be used for application development in an SDK. Check with your SDK provider to see if these are available. CMake is an open-source, cross-platform family of tools which can be used with the SDK to build, test and package software. If the generated VxWorks SDK was built with CMake enabled, when you source the SDK environment, you will automatically have access to CMake. There is no need to download, install and configure Cmake separately. With VxWorks CMake, you can build existing CMake examples that are included in the generated SDK, or you can create your own projects.
Application type	Kernel applications (DKMs) User applications (RTPs)	Check with your SDK provider to see which of these are available.
Documentation	VxWorks Kernel API References VxWorks Application API References	If your installed SDK includes VxWorks reference documentation, use your browser to launch: *sdkInstallDir* **/docs/vxworks-7/html/doc_index.htm**

3. PREPARING VISUAL STUDIO TO USE A VXWORKS SDK

The application developer can optionally use Visual Studio (VS) with a VxWorks software development kit (SDK).

Before You Begin

- Ensure that you have all the prerequisite information.

Procedure

1. Install the Visual Studio Code (VSCode) IDE for your operating system.
   https://code.visualstudio.com/download
2. Launch VSCode and install the VxWorks SDK extension from the Visual Studio Marketplace.
   https://marketplace.visualstudio.com/items?itemName=WindRiver.VxWorksSDKSupport

   The VxWorks SDK extension supports several project types. You will need to reference your SDK from the extension settings.

3. Browse to the VxWorks SDK Support extension in your VSCode Extension listing.

   Click on the settings "gear icon" and choose **Extension Settings**.

   The extension allows referencing the SDK on a user or workspace basis. Add the top-level location of your SDK (*sdkInstallDir*) to both.

4. Navigate to or create your workspace.

   To create your first workspace:

   1. Select **File > Open Folder** and browse to the location for your VxWorks Application projects.
   2. To make the folder a VSCode Workspace, use **File > Save Workspace As** and give the workspace a name.

   You can now create, build and debug application projects in the workspace you have created. Any projects created while this workspace is active will be placed within the workspace folder. Right clicking on this workspace will give you options to create VxWorks projects in this directory.

4. DEVELOPING APPLICATIONS USING VSCODE

As an application developer, you can use Visual Studio Code (VSCode) to create, build, and debug your applications.

Before You Begin

- Set up VSCode for use with your SDK; see Preparing Visual Studio to Use a VxWorks SDK on page 3.
- If you require QEMU and it is not included in your SDK, install QEMU; see *VxWorks 7 SDK QEMU Installation Guide*.
- Python is used by the helper script provided to start a QEMU virtual target. If you require QEMU and python is not included in your SDK, determine the version of python you need.
  - If the default on your host is version 2*x*, use the python3 command.

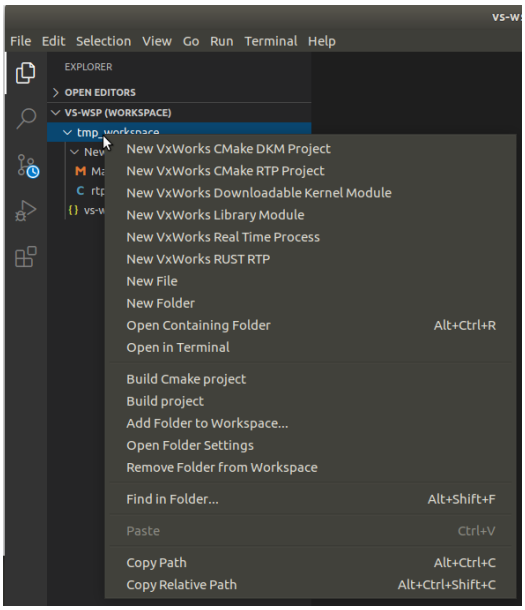
Procedure

1. If required, install or access at least version 3.6 of python.

Host Type	Instructions
Linux	To install directly on the host: `$ sudo apt-get install python3 && python3 -m pip install -U pip`
Windows	To install directly on the host: Download for Windows 10 from the following, ensuring you select **Add Python to PATH**: https://www.python.org/downloads/

2. Create a project with VSCode.

   Right-click in the workspace folder and select the project type (**New VxWorks ...**).



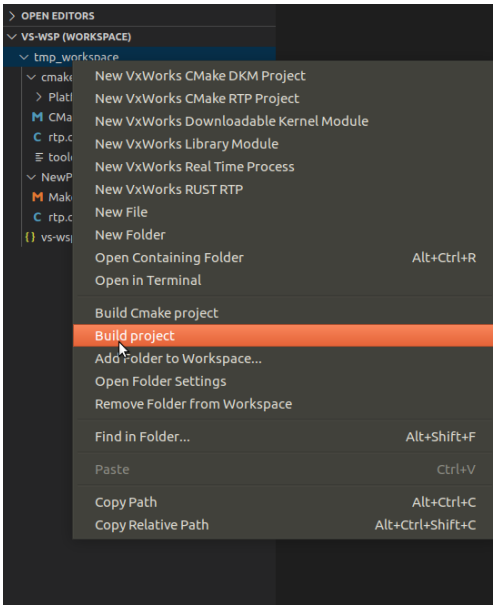
3. In the edit box that appears, enter the name of your project.

   VSCode automatically creates a folder, a top-level C file (with "Hello World" code) and a Makefile for the project you created.

   From this dialog you are able to create C-code projects that use traditional makefiles, CMake tooling, or a Rust-language project (if supported by your VxWorks SDK).

4. Create and edit your source files.
5. Build the project using VSCode.

   Right-click the project name and select **Build project** or **Build Cmake project**.



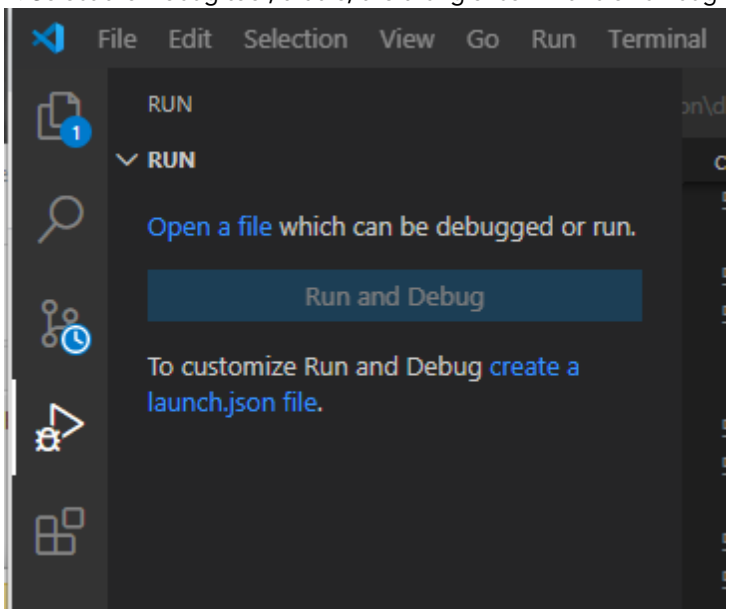
   After your build succeeds, the typical scenario is to load the binary on a running target, debug any issues, iterate as required until you achieve successful execution.

6. Ensure the target is running the kernel image that came with your SDK.

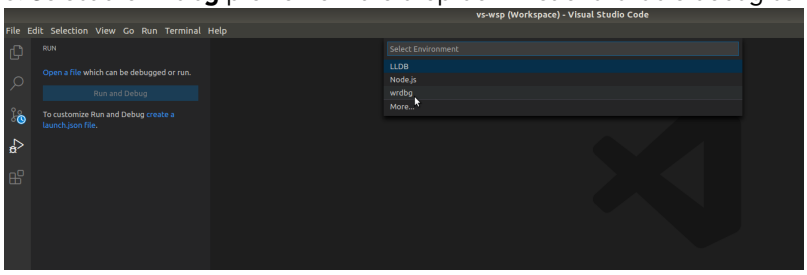
   See the *VxWorks 7 SDK Boot Guide*.

   You can try the **startGenericQemu** setting in the **launch.json** to run a default QEMU-enabled image based on the same architecture as the target SDK VxWorks image. Note that a given target VxWorks image may not work with the generic QEMU.

7. Create a debug configuration (if one does not yet exist).
   1. Select the Debug tool; that is, the triangle icon with a small bug in the lower left corner (see below).



   2. Click **create a launch.json file**.
   3. Select the **wrdbg** profile from the drop-down list of available debug configuration options.



   The **launch.json** configuration file will appear. For example, for an RTP with QEMU, the default file is similar to the image below.

```
.vscode > {} launch.json > ...
1  {
2      // Use IntelliSense to learn about possible attributes.
3      // Hover to view descriptions of existing attributes.
4      // For more information, visit: https://go.microsoft.com/fwlink/?link
5      "version": "0.2.0",
6      "configurations": [
7          {
8              "name": "Run RTP w/ QEMU",
9              "type": "wrdbg",
10             "request": "launch",
11             "target": "<RTP.VXE_FILE>",
12             "cwd": "${workspaceRoot}/<RTP_DIRECTORY>",
13             "valuesFormatting": "parseText",
14             "printCalls": false,
15             "sdkPath": "${config:vscode.vxsdk}",
16             "targetType": "rtp",
17             "startQemu": true,
18             "stopQemu": true,
19             "noDebug": true,
20             "trace": false
21         },
```

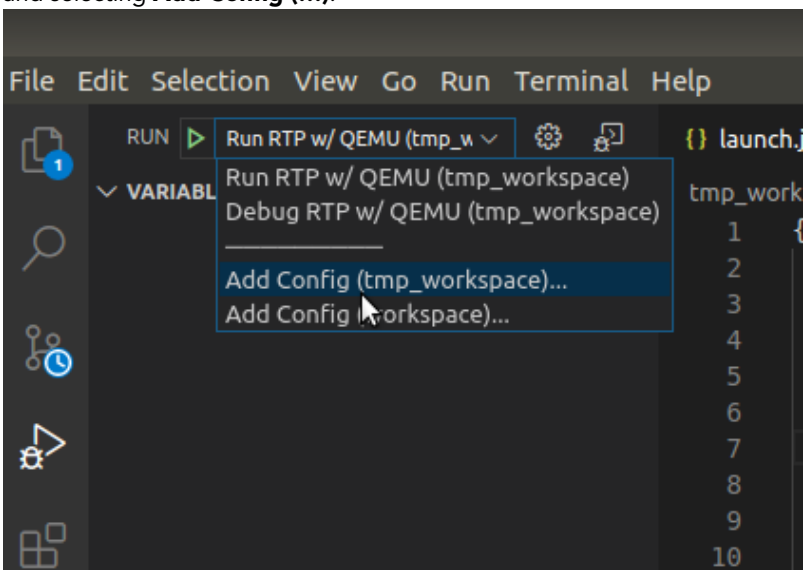
4. To run or debug an RTP, the parameter values that contain placeholders must be updated.

   In the screenshot from the previous step:

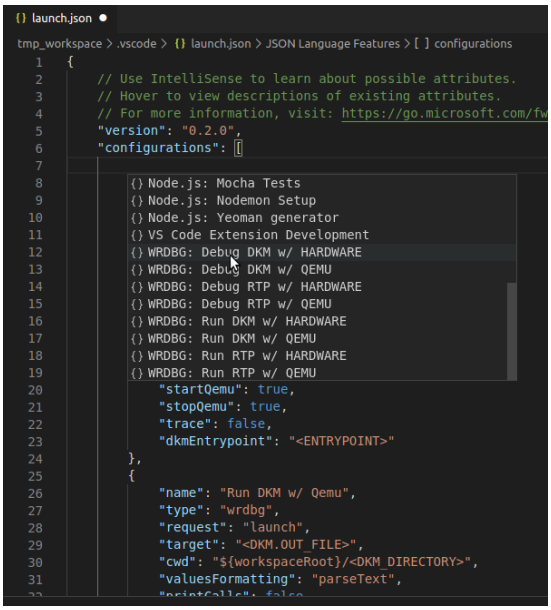
   - Replace the placeholder value of **target** (**<RTP.VXE_FILE>**) with the **.vxe** file name for your application.
   - Replace the placeholder value of **cwd** (**<RTP_DIRECTORY>**) with your project folder name.

   For other options, see VSCode Debug Configuration Settings on page 9.

8. As required, add debugging configurations with different default settings by clicking on the drop-down box in the Debug window and selecting **Add Config (...)**.



The list of options below will appear in your **launch.json** file.



9. Choose one of the options prefixed with **WRDBG:**

   Note that the only difference between the **Run** and **Debug** project configurations is the value set for the **noDebug** parameter in that section.

10. To run/debug an application, press the green side-arrow icon to start the debugging session.

    The VSCode interface provides a range of debugging features allowing you to add breakpoints, step in and out of functions, view symbols, stack traces, process states, and more. Refer to the debugging section of the *VxWorks 7 SDK Visual Studio Code Guide* for more information.

5. VSCODE DEBUG CONFIGURATION SETTINGS

These debug configurations are preset to enable execution of DKM applications or RTP applications from VSCode using the debugger.

Name (with Type)	Required or Optional (with default)	Description
cwd (string)	Required	Specifies the path to your project.
sdkPath (string)	Required	Specifies the local path to the SDK.
target (string)	Required	Specifies the path to your application executable.
targetType (string)	Required	Specifies the target type of the VxWorks executable (RTP or DKM). This is technically not required if **target** has a valid file extension (**.vxe** for RTPs and **.out** for DKMs).
arguments (string)	Optional	Specifies arguments to append after the executable. You can also use pipes.
autorun (string)	Optional	Specifies GDB commands to run when starting to debug.
debugger_args -debug (array)	Optional	Specifies additional arguments to pass to GDB. These are functionally executable arguments so the option and the value must be separate array entries; for example, ["-ip", "xxx.xxx.xxx.xxx"]. If the -debug option is specified, this enables debug output at the interpreter level.
dkmEntrypoint (string)	Optional (none)	Specifies the name of the DKM function that will be the debugging entry point. This is technically optional only for non-DKMs. It is required with DKMs.
env (object)	Optional	Specifies an environment overriding the gdb (and, in turn, also the process) environment.
gdbpath (object)	Optional (points to the Wind River wrdbg utility)	Specifies the path to the gdb executable or the command if accessible in the host's **PATH** environment variable. This should not be changed as any other GDB-equivalent utility will not be able to debug or run RTPs and DKMs in a VxWorks environment.
noDebug (boolean)	Optional (False)	Specifies if debugging features are enabled (True) or not (False). If False, you can only execute the application.
startGenericQemu (boolean)	Optional (False)	Specifies if the system automatically enables starting a generic QEMU image for each debug session.

Name (with Type)	Required or Optional (with default)	Description
stopQemu (boolean)	Optional (False)	Specifies if the QEMU instance should be terminated after debugging session has completed.
targetIp (string)	Optional (localhost)	Specifies the IP address of target board.
terminal (string)	Optional	If not set, this keeps program output in the VSCode console at the bottom. If set to the empty string on a Linux host, the program will spawn in a new console using **x-terminal-emulator**. Otherwise, on a Linux host, spawns the program with the specified terminal. If set to the empty string on a Windows host, spawns the program in a new console (no other console is supported).
valuesFormatting (string)	Optional	Specifies how to show variable values: **disabled: show value as is** **parseText: parse debuggers output text into structure** **prettyPrinters: enable debugger's custom pretty-printers, if there are any**
vxWorksKernelImage (string)	Optional (*sdkInstallDir* **/bsps/** *bspName* **/boot/vxWorks**)	Specifies the local path of the VxWorks kernel image.

6. DEVELOPING APPLICATIONS USING THE CLI

As an application developer, you can use the command line interface (CLI) to create, build, and debug your applications.

Before You Begin

- See Application Developer Prerequisites on page 2.
- If you require QEMU and it is not included in your SDK, install QEMU; see *VxWorks 7 SDK QEMU Installation Guide*.
- Python is used by the helper script provided to start a QEMU virtual target. If you require QEMU and python is not included in your SDK, determine the version of python you need.
  - If the default on your host is version 2*x*, use the python3 command.

Procedure

1. If required, install or access at least version 3.6 of python.

Host Type	Instructions
Linux	To install directly on the host: `$ sudo apt-get install python3 && python3 -m pip install -U pip`
Windows	To install directly on the host: Download for Windows 10 from the following, ensuring you select **Add Python to PATH**: https://www.python.org/downloads/

2. Create a project workspace using the CLI.

Host Type	Instructions

Linux	In a preferred location (such as your home directory):
	`$ mkdir my-vxworks-project`
	`$ cd my-vxworks-project`

Windows	In a preferred location (such as your home directory):
	`% mkdir my-vxworks-project`
	`$ cd my-vxworks-project`

3. Create a C-source file in your project folder.
4. If you are using traditional Makefile builds, do the following:

Host Type	Instructions
Linux	`$ cp sdkInstallDir/examples/makefiles/appType/generic_appType_makefile Makefile`
Windows	`$ copy sdkInstallDir\examples\makefiles/appType\generic_appType_makefile Makefile`

5. If you are using CMake Toolchain builds, do the following:

Host Type	Instructions
Linux	Create the **CmakeLists.txt** file in your project folder. `$ cp sdkInstallDir/cmakefiles/appType/appType_toolchain.cmake toolchain.cmake`
Windows	Create the **CmakeLists.txt** file in your project folder. `$ copy sdkInstallDir\cmakefiles\appType\appType_toolchain.cmake toolchain.cmake`

6. Build your application.

Makefile Type	Instructions
Traditional	For any host type: `$ make`
CMake	For any host type: `$ cmake -DCMAKE_TOOLCHAIN_FILE=toolchain.cmake .`

7. If not already running, boot the kernel image from your SDK on the target.

   See *VxWorks 7 SDK Boot Guide*.

8. Connect to the target with wrdbg.

   See Connecting to a Target With wrdbg on page 14.

9. Load the application onto the target.

Application Type	Instructions

RTP	For any host type, and for example, your application prints **"Hello world!"**:
	`(wrdbg) file RTPapplication.vxe`
	`Reading symbols from RTPapplication.vxe...done`
	`[New Thread 1]`
	`[New inferior 2 (0xffff800000534040)]`
	`Switching to inferior 2 [0xffff800000534040] (RTPapplication.vxe)`
	`(wrdbg) run`
	`Starting program: RTPapplication.vxe`
	`_RTP_OUTPUT_START_`
	
	`Hello World!`
	
	`[Inferior 2 (process 0xffff800000534040) exited]`
	`_RTP_OUTPUT_END_`
	
	`(wrdbg)`

DKM	For any host type, and for example, you are using the built-in DKM example that comes with your SDK:
	`(wrdbg) module load hello_world.out`
	`Loading module DKMapplication.out...done`
	`Module id = 0xffff8000005146c0`
	
	`(wrdbg)`
	
	From the VxWorks kernel shell:
	`-> startHelloWorld`
	`DKM Hello World!`
	
	`->`

For more information about using wrdbg, see:

sdkInstallDir **/toolkit/wrdbg_tools/lib/python/doc/html/wrdbg/wrdbg.html**

7. CONNECTING TO A TARGET WITH WRDBG

When the VxWorks image provided with a software development kit (SDK) is configured with a debug agent, you can connect over a network-based target communication framework (TCF) connection using the wrdbg toolset.

Before You Begin

- You have already booted the VxWorks image on the target.
- The network interface on the target must be up and running, with network IP addressing that aligns with your network administrator to enable connectivity to the target.

Procedure

1. From the VxWorks kernel shell (serial console of the target), determine the assigned IP address of the target.

   When your image includes the ifconfig utility, the IP address is shown next to the **inet** keyword (in this example, 10.10.1.2).

```
-> ifconfig
...
gei0 Link type:Ethernet  HWaddr 00:0a:35:04:9a:50
   capabilities: TXCSUM VLAN_MTU
   inet 10.10.1.2 mask 255.255.255.0  broadcast 10.10.1.1
   inet6 unicast fe80::20a:35ff:fe04:9a50%gem0  prefixlen 64  automatic
   UP RUNNING SIMPLEX BROADCAST MULTICAST
   MTU:1500  metric:1  VR:0  ifindex:2
   RX packets:95618 mcast:0 errors:0 dropped:1246
   TX packets:63535 mcast:5 errors:0
   collisions:0 unsupported proto:0
   RX bytes:11095185 (11 M)  TX bytes:6620982 (6620 k)
```

2. In a host terminal window, set up the SDK environment variables.

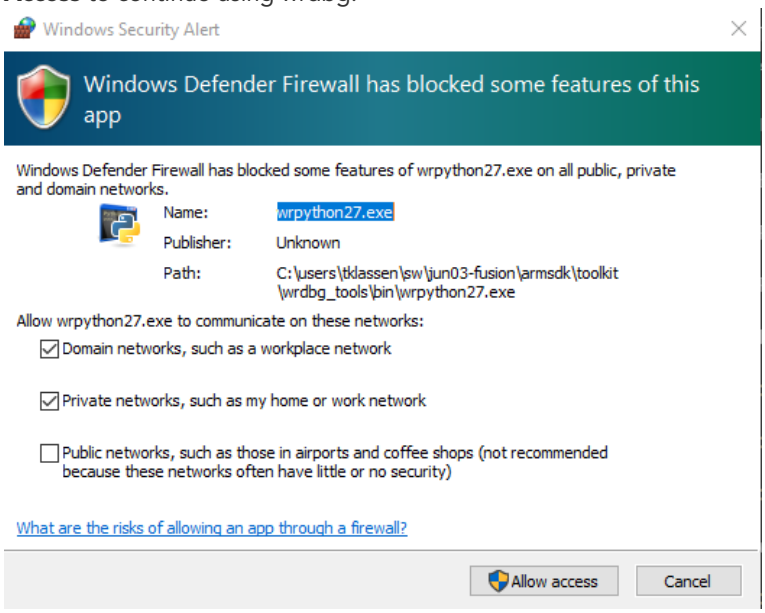
Host Type	Instructions
Linux	`$ source` *sdkInstallDir* `/toolkit/wind_sdk_env.linux`
Windows	`$` *sdkInstallDir* `\toolkit\wind_sdk_env.bat`

3. Invoke the Wind River debugger (wrdbg).

```
$ wrdbg
Debug Shell, 4.18

Copyright (C) 2014-2020 Wind River Systems, Inc.

Refer to the Core documentation of WRDBG (file://sdkInstallDir/toolkit/wrdbg_tools/lib/python/d
oc/html/wrdbg/wrdbg.html)

Type "help" to get information about commands.
```

```
(wrdbg)
```

Note: On a Windows host, you may see the dialog below as it relies on an included python interpreter. Select **Allow Access** to continue using wrdbg.



4. Connect to the target.

   This example assumes you are using QEMU on your host with an Intel target image.

```
(wrdbg) target connect vxworks7:TCP:10.10.1.2:1534 -kernel bsps/itl_generic_BSPversion/boot/vxWorks
Connecting...
Connected to TCP:10.10.1.2:1534

(wrdbg) info target
Connection                Name                 User       OS
---------------------+--------------+----------+---------------------
TCP:10.10.1.2:1534    VxWorks 7      target     VxWorks 7 SMP 64-bit
TCP:127.0.0.1:1534    VxWorks 7      target     VxWorks 7 SMP 64-bit

(wrdbg)
```

5. If target connectivity is lost, the wrdbg commands will fail, often with a response similar to the following:

```
Error: ** Command aborted: Target connection is down. **
```

   This is typically because the running image has failed or been aborted.

6. Boot the VxWorks image again.
7. In the wrdbg shell, if you try to re-issue the target connect, it may respond with:

```
Already connected to "TCP:10.10.1.2:1534".
```

However, this will not likely allow you to continue using wrdbg.

8. The recommended procedure is to first disconnect and then connect again.

```
(wrdbg) target disconnect
Disconnected from localhost:1534
(wrdbg) target connect vxworks7:TCP:10.10.1.2:1534 -kernel bsps/itl_generic_BSPversion/boot/vxWor
ks
...
```